HealthNest

CS673 Software Engineering | Team 2 | Final Iteration

Chamarr Auber, Aaron Brown, Deven Shah, Stephanie Wang

What We’ll Cover

01

Team & Project Overview

Roles, vision, tech stack, journey across iterations

02

Requirements Analysis

Functional & non-functional requirements, user stories, traceability

03

Design

Architecture, database, design patterns, REST APIs, Pulse AI / RAG

04

Implementation

Feature progression, Iteration 3 deep dive, dev practices

05

Testing

Strategy, metrics, growth, test case examples, known issues

06

CI/CD & Security

Pipeline, git workflow, security design, AI across the SDLC

07

Project Management

Contributions, risk management, achievements

08

Final Status & Live Demo

End-to-end walkthrough, future work

Meet the Team

Chamarr Auber

Team Lead

Configuration Lead

Deven Shah

Team Lead

Requirements Lead

Aaron Brown

Security Lead

Design & Implementation Lead

Stephanie Wang

QA Lead

Design & Implementation Lead

What is HealthNest?

HealthNest is an AI-powered healthcare platform that brings appointments, lab results, secure messaging, and care-team coordination into one place for patients and providers — with Pulse AI assisting both sides.

- **Frontend** — React 19 + Vite SPA
- **Backend** — Python FastAPI REST API
- **Database / Auth** — Supabase (PostgreSQL) + local Postgres 16 for dev
- **AI Layer** — OpenAI GPT-4o — Pulse AI (Patient & Doctor)
- **Infrastructure** — Docker Compose, GitHub Actions CI/CD

Our Journey

Iteration 0

Planning, SPPP, tech stack, roles, risk register

Iteration 1

Login, dashboards (mock data), lab results backend, CI/CD

Iteration 2

Appointments live, Lab Results full workflow, Pulse AI (patient)

Iteration 3

Secure Messaging, Care Team, DFA skills, biometric prototype, security hardening

Functional Requirements Overview

Patient

- Sign up / sign in with patient role
- View Patient Dashboard
- Book / reschedule / cancel appointments
- Released lab result view
- Care team / provider view
- Pulse AI Patient-Facing Assistant
- Secure messaging
- Patient account settings

Provider

- Sign up / sign in with provider role
- View Doctor Dashboard & daily schedule
- Manage provider schedule and appointment slots
- Lab result upload, review, edit, and release workflow
- Patient lookup / patient context view
- Pulse AI Doctor-Facing Assistant
- DFA visit overview / encounter note workflow
- Provider account settings

Shared System

- Role-based authentication
- Supabase-backed session management
- Shared account settings
- Passkey / biometric prototype
- Secure messaging infrastructure
- Connected appointment and lab data
- Shared React + FastAPI API layer
- AI assistant backend integration
- Automated frontend and backend testing

Non-Functional Requirements

Beyond features, HealthNest's design is shaped by quality attributes that matter for a healthcare application.

Security & Privacy

Sensitive health data is protected through Supabase authentication, role-based access, environment-based secrets, and controlled patient/provider workflows. Lab results are only visible to patients after provider release.

Usability

Dashboards are designed so patients and providers can reach key information with minimal clicks, using consistent layout, typography, and role-specific workflows.

Reliability

Automated frontend and backend tests help catch regressions as features are added. CI/CD support helps identify build, lint, and test failures before merging.

Maintainability

The system uses a layered backend, reusable frontend components, shared API helpers, and feature-based PRs to keep the codebase easier to review and extend.

Performance

Pulse AI uses streaming-style responses where supported, and frontend helper functions keep the interface responsive during chat, dashboard, and appointment workflows

Compliance (HIPAA-aware)

The design is HIPAA-aware. HealthNest separates patient and provider access, limits lab visibility through release workflows, and treats health data as sensitive throughout the application.

Requirements Tracking & Traceability

Jira Tickets

Each feature was tracked as a Scrum ticket

Feature Branches & PRs

Implementation work was connected to branches and pull requests

Testing Evidence

Completed features were verified through automated and manual testing

How we track requirements

Jira ticket → feature branch → pull request → review/merge → automated or manual testing

Major Requirement Flows

Requirement Area	Example Ticket	User Story	Status
Authentication & Role-Based Access	SCRUM-7, SCRUM-56	As a patient or provider, I can sign up, sign in, and be routed to the correct role-based experience.	Done
Patient Appointment Workflow	SCRUM-5	As a patient, I can find an available provider, book an appointment, and later reschedule or cancel it.	Done
Lab Results Release Workflow	SCRUM-61	As a provider, I can upload, review, edit, and release lab results so that patients can securely view finalized results.	Done
Patient Care Team & Secure Messaging Flow	SCRUM-75, SCRUM-6, SCRUM-54, SCRUM-78	As a patient or provider, I can view care-team contacts and send secure messages within an approved care relationship.	Done
Pulse AI Patient & Provider Assistant Flow	SCRUM-39, SCRUM-27	As a patient or provider, I can ask Pulse AI role-specific questions and receive assistant support for my healthcare workflow.	Done
Doctor-Facing Visit Overview	SCRUM-58, SCRUM-80	As a provider, I can view patient context, visit details, and encounter notes to prepare for a patient visit.	Done

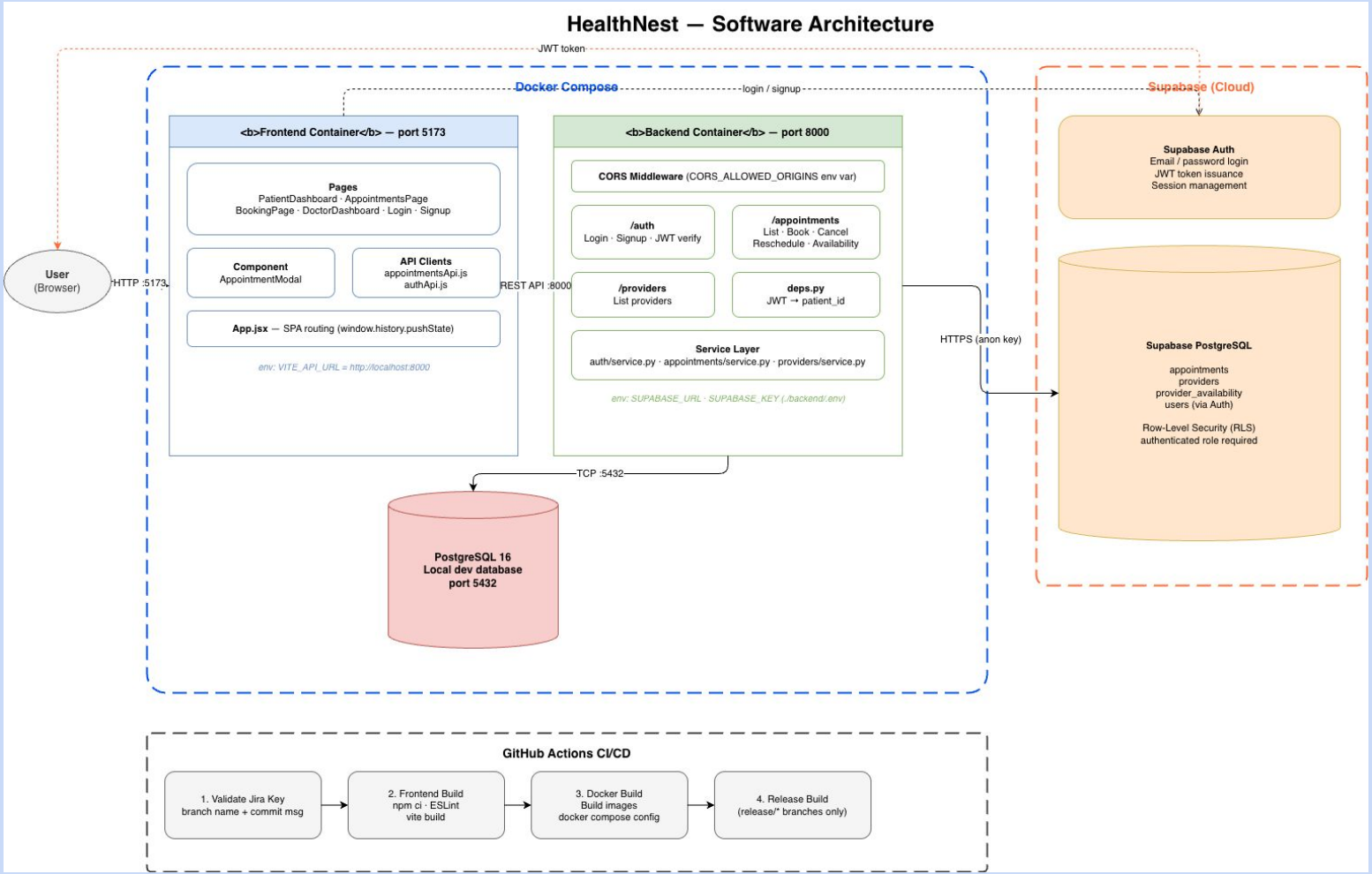
Architecture

Frontend

- SPA on dedicated Docker container
- Rendering user interface
- Handling client-side routing
- Communicating with backend through REST API
- Used React for reusable UI components

Backend

- Python FastAPI hosted on dedicated Docker container
- Middleware layer handles CORS configuration
- Service architecture in which route handlers delegate business logic to dedicated service modules


HealthNest — Software Architecture
JWT token
Docker Compose
login / signup
Supabase (Cloud)
Frontend Container — port 5173
Pages
PatientDashboard · AppointmentsPage
BookingPage · DoctorDashboard · Login · Signup
Component
AppointmentModal
API Clients
appointmentsApi.js
authApi.js
App.jsx — SPA routing (window.history.pushState)
env: VITE_API_URL = http://localhost:8000
Backend Container — port 8000
CORS Middleware (CORS_ALLOWED_ORIGINS env var)
/auth
Login · Signup · JWT verify
/appointments
List · Book · Cancel
Reschedule · Availability
/providers
List providers
deps.py
JWT → patient_id
Service Layer
auth/service.py · appointments/service.py · providers/service.py
env: SUPABASE_URL · SUPABASE_KEY (./backend/.env)
User
(Browser)
HTTP :5173
REST API :8000
HTTPS (anon key)
TCP :5432
PostgreSQL 16
Local dev database
port 5432
Supabase Auth
Email / password login
JWT token issuance
Session management
Supabase PostgreSQL
appointments
providers
provider_availability
users (via Auth)
Row-Level Security (RLS)
authenticated role required
GitHub Actions CI/CD
1. Validate Jira Key
branch name + commit msg
2. Frontend Build
npm ci · ESLint
vite build
3. Docker Build
Build images
docker compose config
4. Release Build
(release/* branches only)

Testing Strategy

Testing Type	Notes
Manual Acceptance Testing	Validated patient/provider workflows using STD test cases and local Docker testing.
Frontend Automated Testing	Jest + React Testing Library verified UI rendering, form validation, role-based routing, dashboards, appointments, lab results, messaging, care team, and Pulse AI components.
Backend Automated Testing	pytest verified API/service behavior for authentication, scheduling, messaging, care team, lab results, Pulse AI, and safety-classifier logic.
Regression Testing	Repeated automated and manual checks were used before merges and final demo preparation to catch broken workflows.
CI/CD Regression Safety Net	GitHub Actions supported build, lint, and test checks to catch failures during pull requests and integration.

Testing Metrics & Growth

31	259	227	486
Manual acceptance test cases	Frontend automated tests passed	Backend pytest tests passed	Total automated tests passed

Final Automated Coverage Areas

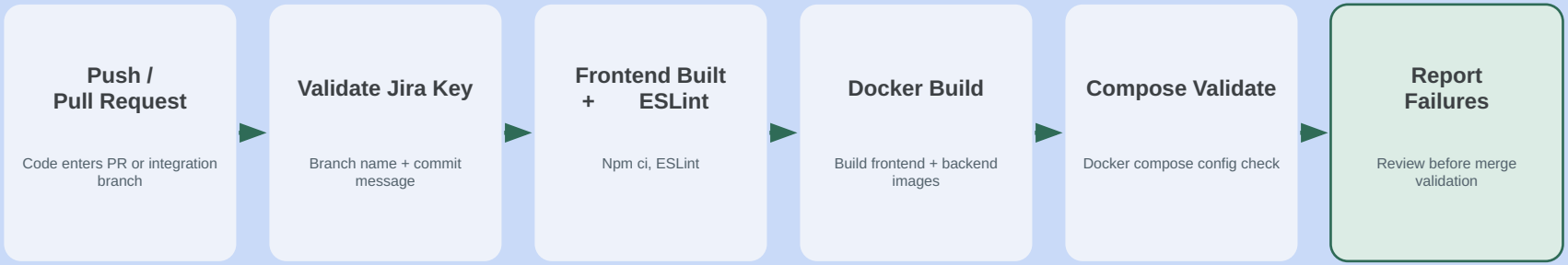
- Frontend tests cover dashboards, appointments, lab results, messaging, care team, Pulse AI, and doctor dashboard workflows
- Backend pytest covers authentication, scheduling, messaging, care team, lab results, Pulse AI pipeline, safety classifier, and provider overview logic
- Final automated test run passed across both frontend and backend

Manual Test Case Example:

Test Case ID	TC11
Name	Provider Uploads a Lab Result File
New or Old	New
Test Items	SCRUM-61 Lab Results — File upload flow for Provider role
Test Priority	High
Dependencies	User must be signed in as Provider; a valid HLT lab result file must be available
Preconditions	Provider is signed in and on the Lab Results page
Input Data	Authenticated Provide session; sample HL7 file
Test Steps	1. Sign in as Provider and navigate to the Lab Results page 2. Click Browse files or drag and drop a lab result file onto the upload area 3. Select a patient from the patient typeahead search 4. Click upload & review
Postconditions	Lab result file is uploaded and parsed; the provider is taken to the review page
Expected Output	File uploads successfully and the lab result appears in the pending results list with status uploaded
Actual Output	File uploaded successfully. Parsed entries appeared in the review form.
Pass or Fail	**PASS**
Bug ID / Link	N/A
Additional Notes	

CI/CD & Automated Testing Integration

Pipeline Flow

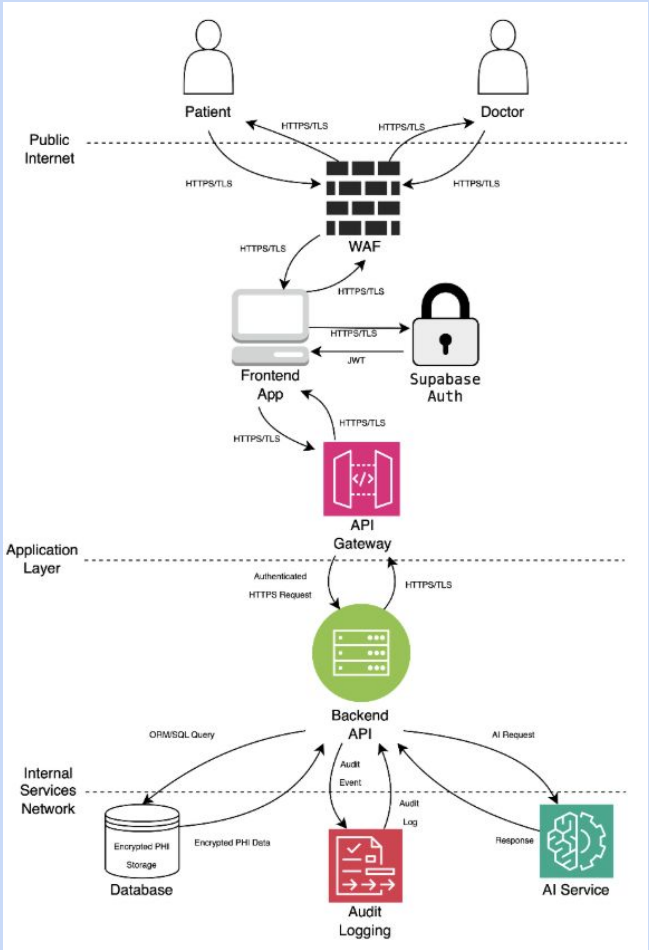


Why this Matters

- Catches regressions early
- Gives fast feedback to developers
- Supports safer pull requests
- Complements manual QA testing

Security Architecture & Goals

- Protect PHI in transit and at rest
- Enforce authentication and RBAC authorization
- Maintain auditability and HIPAA awareness
- Mitigate OWASP Top 10 security risks



Security Testing

SAST

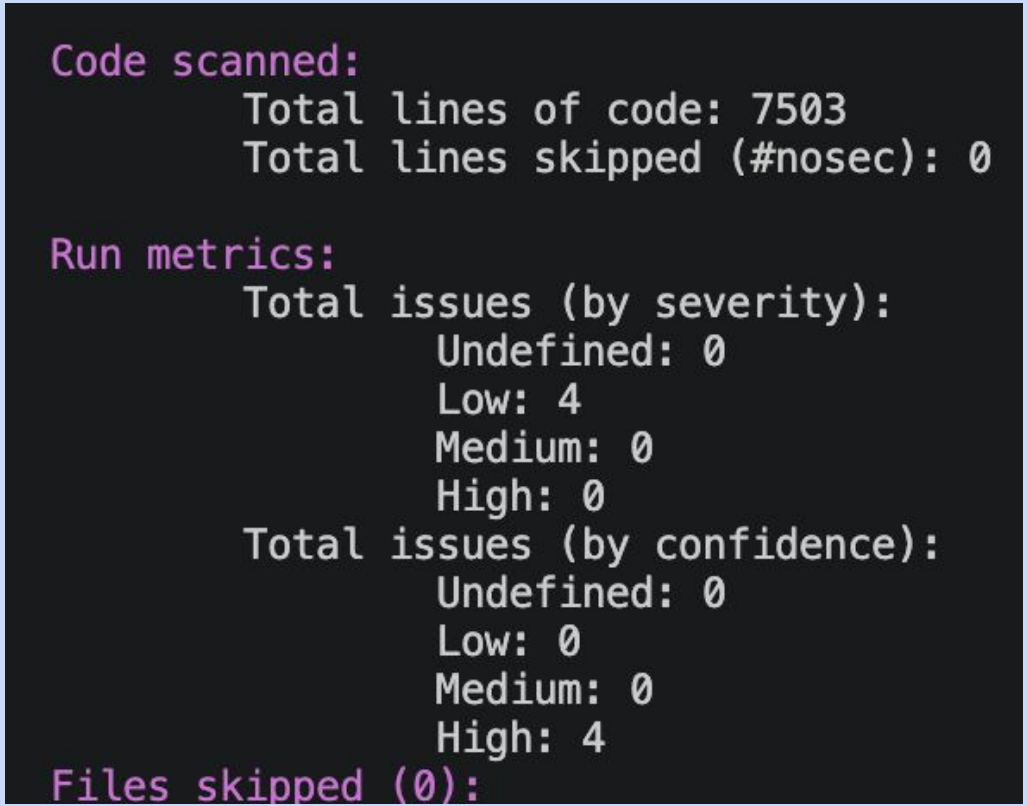
- Bandit
- SonarQube

DAST

- ZAP
  - Unauthenticated Active Scanning
  - Authenticated patient/provider application exploration
  - Planned: Cross-account authorization/IDOR testing

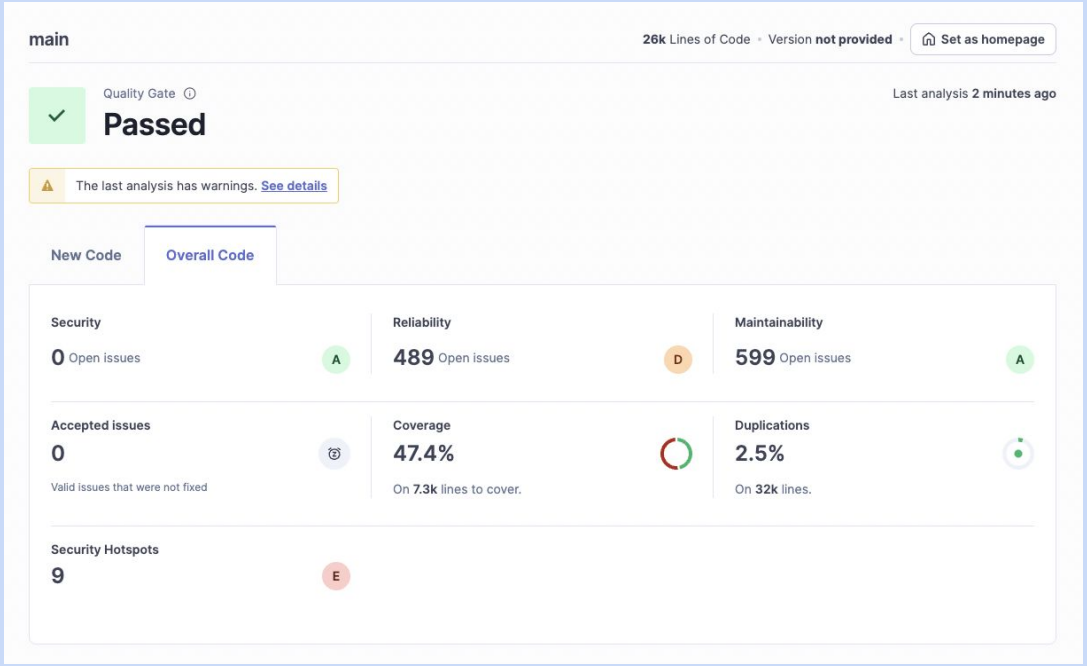
Bandit

- 7,503 lines of code scanned
- 0 medium/high severity findings
- 4 low severity findings reviewed

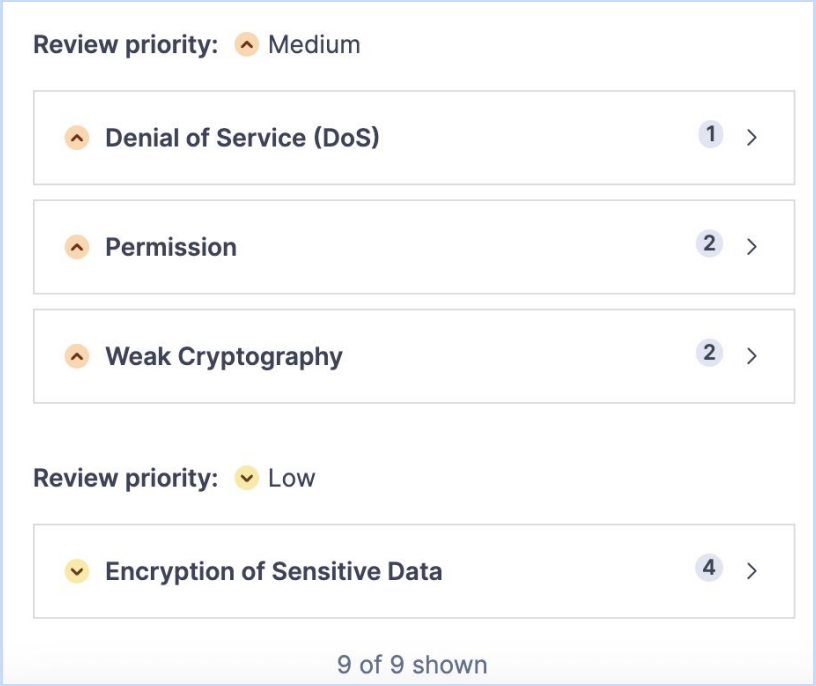


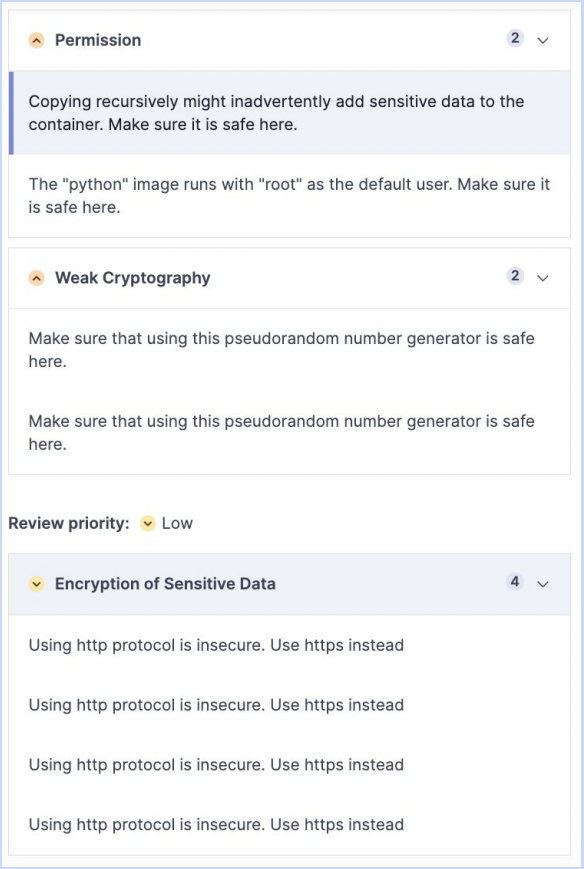
SonarQube

- Passed Quality Gate
- 0 security vulnerabilities
- 9 security hotspots identified



Security Hotspots





main

26k Lines of Code • Version not provided • Set as homepage

Quality Gate

Passed

Last analysis 1 minute ago

The last analysis has warnings. See details

New Code | Overall Code

Security

0 Open issues — A

Reliability

487 Open issues — D

Maintainability

597 Open issues — A

Accepted issues

0

Valid issues that were not fixed

Coverage

47.9%

On 7.3k lines to cover.

Duplications

2.5%

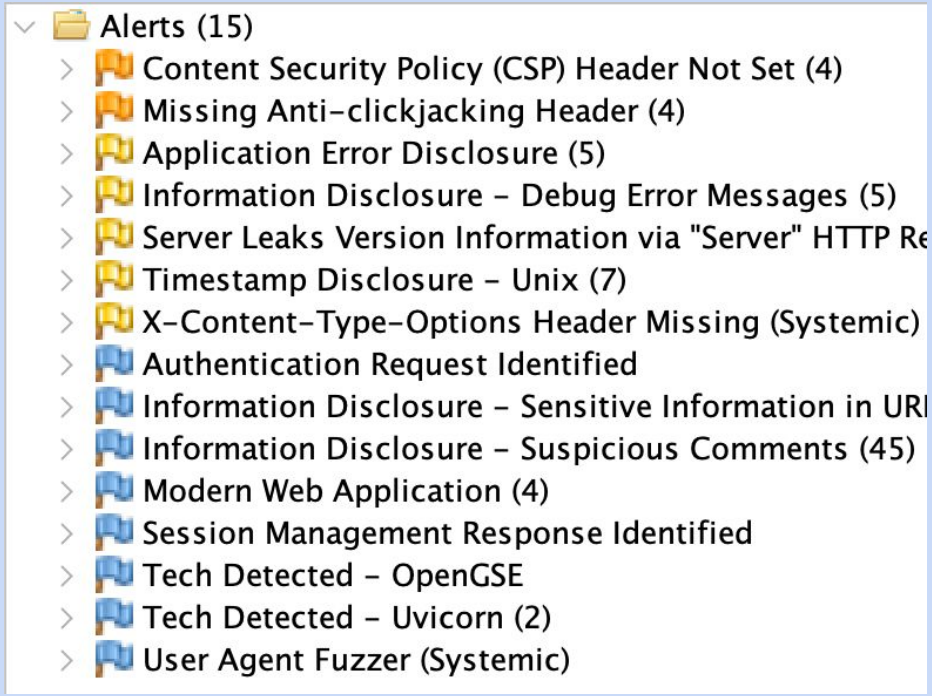
On 32k lines.

Security Hotspots

0 — A

ZAP

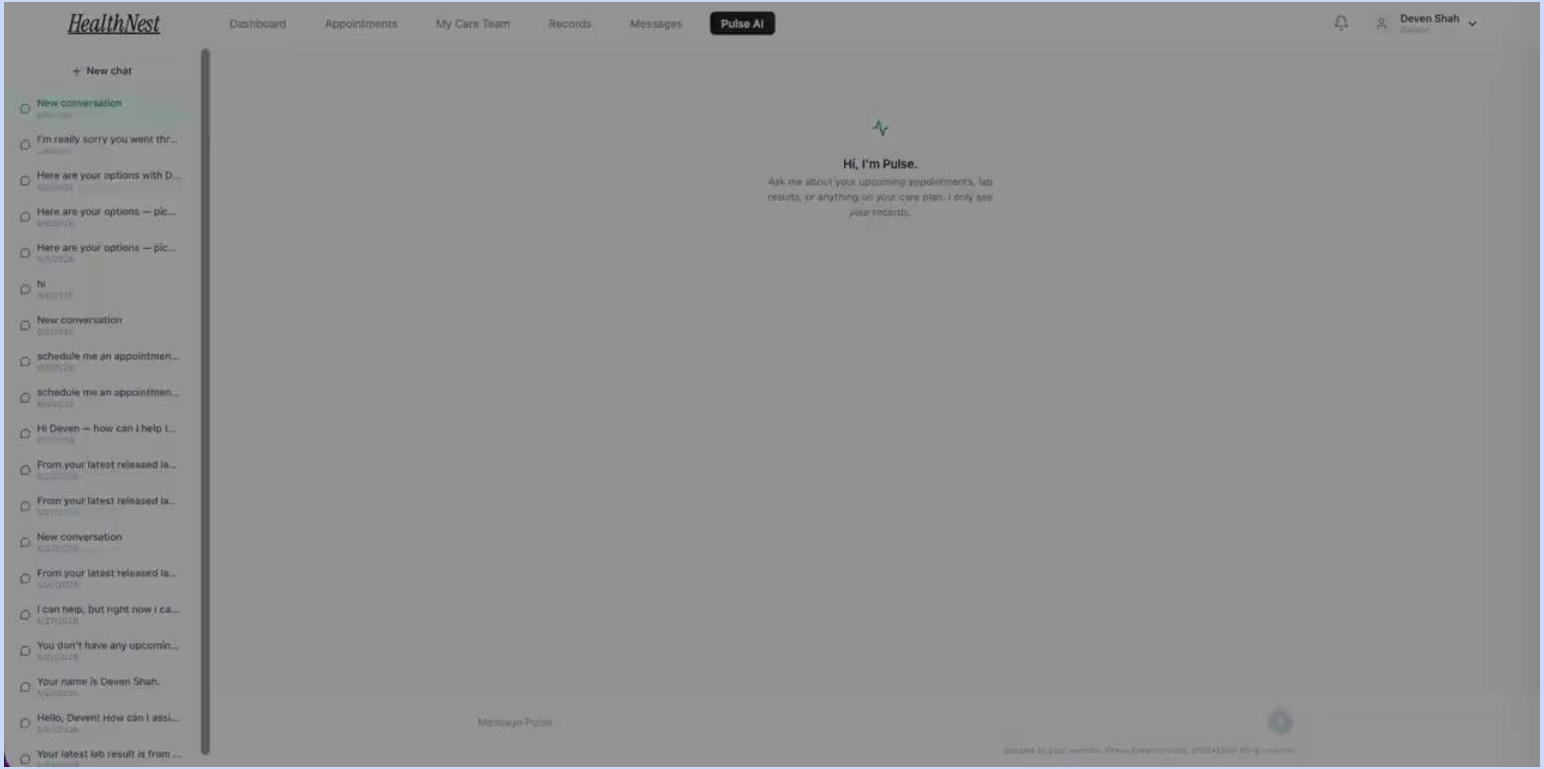
- Missing CSP headers
- Missing anti-clickjacking headers
- Information disclosure findings
- No critical vulnerabilities identified



Additional Planned Security Testing

- **Snyk Dependency Scanning**
- **GitLeaks Secrets Detection**
- **Manual Authorization Matrix**
- **Trivy Container Scanning**

Demo: Pulse


HealthNest
Dashboard
Appointments
My Care Team
Records
Messages
Pulse AI
Deven Shah
+ New chat
New conversation
I'm really sorry you went thr...
Here are your options with D...
Here are your options — pic...
Here are your options — pic...
hi
New conversation
schedule me an appointmen...
schedule me an appointmen...
Hi Deven — how can I help t...
From your latest released la...
From your latest released la...
New conversation
From your latest released la...
I can help, but right now I ca...
You don't have any upcomin...
Your name is Deven Shah.
Hello, Deven! How can I assi...
Your latest lab result is from ...
Hi, I'm Pulse.
Ask me about your upcoming appointments, lab results, or anything on your care plan. I only see your records.
Message Pulse

Thank you!